

Лабораторная работа 4 **КОНТРОЛЬ ВВОДИМЫХ ДАННЫХ И НАСТРОЙКА ИХ ОТОБРАЖЕНИЯ**

Цель работы: ознакомиться с аспектами реализации бизнес-логики на основе триггеров, ознакомиться с форматированием полей и проверкой введенного в поле значения; ознакомиться со свойствами, методами и событиями для фильтрации записей; разработать приложение с контролем вводимых данных и возможностями фильтрации записей, а также поиска в наборах данных.

ПОРЯДОК ВЫПОЛНЕНИЯ РАБОТЫ

1. Ознакомиться с теоретическими сведениями.
2. Создать необходимые триггеры и проверить их работоспособность.
3. Открыть копию проекта лабораторной работы №3.
4. Реализовать триггеры в рамках бизнес-логики
5. Организовать программную проверку вводимых значений при помощи событий **OnSetText**, **OnValidate**, **OnChange** и **BeforePost**.
6. Реализовать изменение отображения данных с использованием события **OnGetText**, свойства **DisplayFormat** и **EditMask**
7. Организовать возможность фильтрации данных на основе **SQL**-запросов.
8. Реализовать фильтрацию по дате с применением свойства **Filter**.
9. Ответить на контрольные вопросы.

ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Триггеры применяются как для обеспечения целостности данных, так и реализации сложной бизнес-логики. В разрабатываемом, в ходе лабораторных работ, приложении бизнес-логика заключается в контроле складских запасов товара в компьютерном магазине. На примере рассмотрим механизм реализации бизнес-логики с использованием триггеров.

4.1 РЕАЛИЗАЦИЯ БИЗНЕС-ЛОГИКИ С ИСПОЛЬЗОВАНИЕМ ТРИГГЕРОВ

В нашем примере при оформлении поставки (поступления товара) необходимо проводить увеличение его количества на складе (изменение значения поля **sCount** таблицы **storage**) на величину, которая указана в поле **ocCount** соответствующей записи таблицы **op_goods_list**. Аналогичная ситуация и в случае оформления продажи – необходимо уменьшить

значение количества товаров на складе. Для этого сначала создадим триггер **tr_op_goods_list_add**, который будет срабатывать после добавления записи в таблицу **op_goods_list** (рис. 4.1):

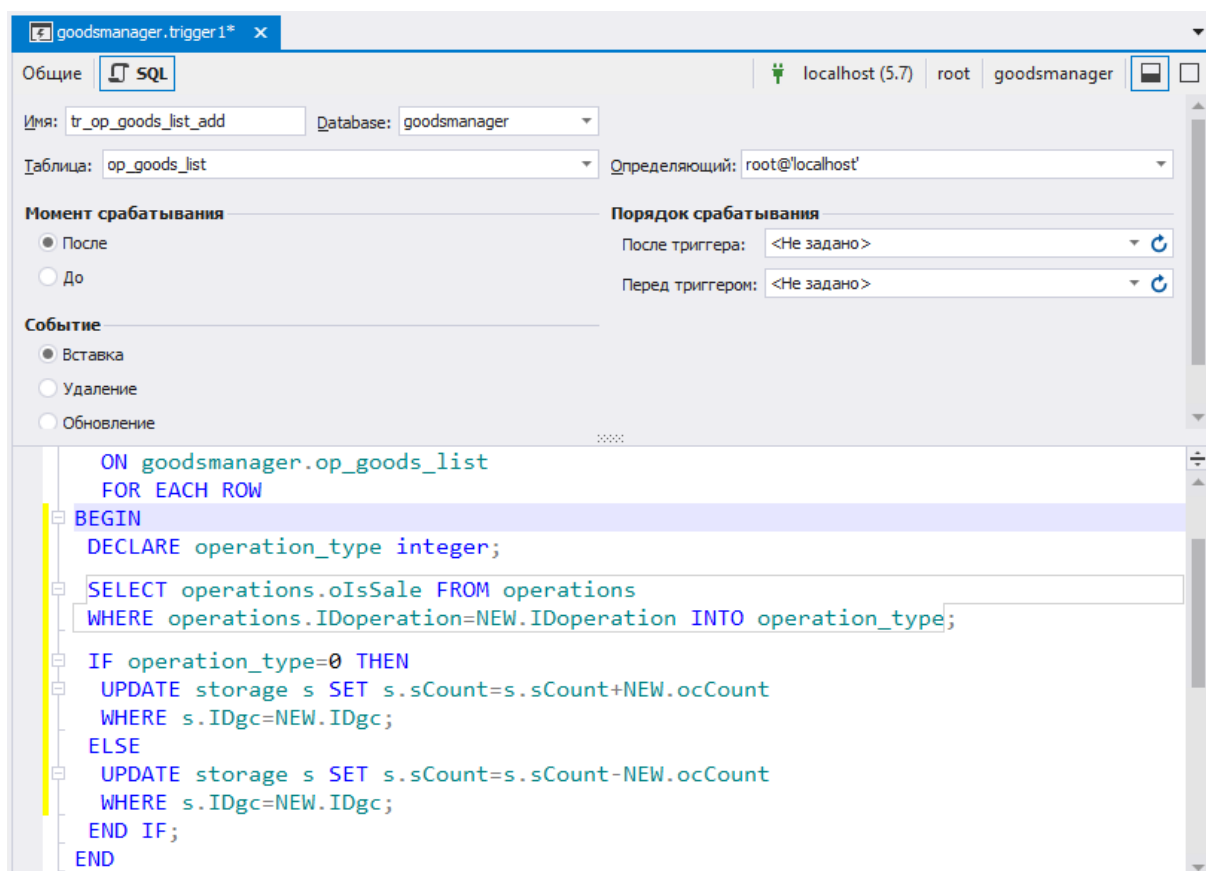


Рис. 4.1. Настройка триггера увеличения/уменьшения количества товаров на складе при добавлении записи в таблицу *op_goods_list*

В данном коде конструкция **DECLARE** отвечает за объявление переменной. При помощи нее объявляется переменная **operation_type**, которая будет определять тип выполняемой операции **0** – поставка, **1** – продажа.

Блок кода **SELECT** отвечает за выборку значения типа операции из базы данных.

При помощи конструкции **IF (условие) THEN действие1 ELSE действие2 END IF** выполняется изменение количества товаров на складе с применением команды **UPDATE (имя таблицы) SET (поле = значение)** с учетом условия для изменения в блоке **WHERE**.

В случае удаления записи из таблицы **op_goods_list** будет использован аналогичный код, но действия в зависимости от типа операции изменятся на противоположные. Настройка триггера **tr_op_goods_list_remove** срабатывающего после удаления записи приведена на рис. 4.2.

Как можно заметить, для обращения к полям удаляемой записи используется псевдоним **OLD** вместо **NEW**.

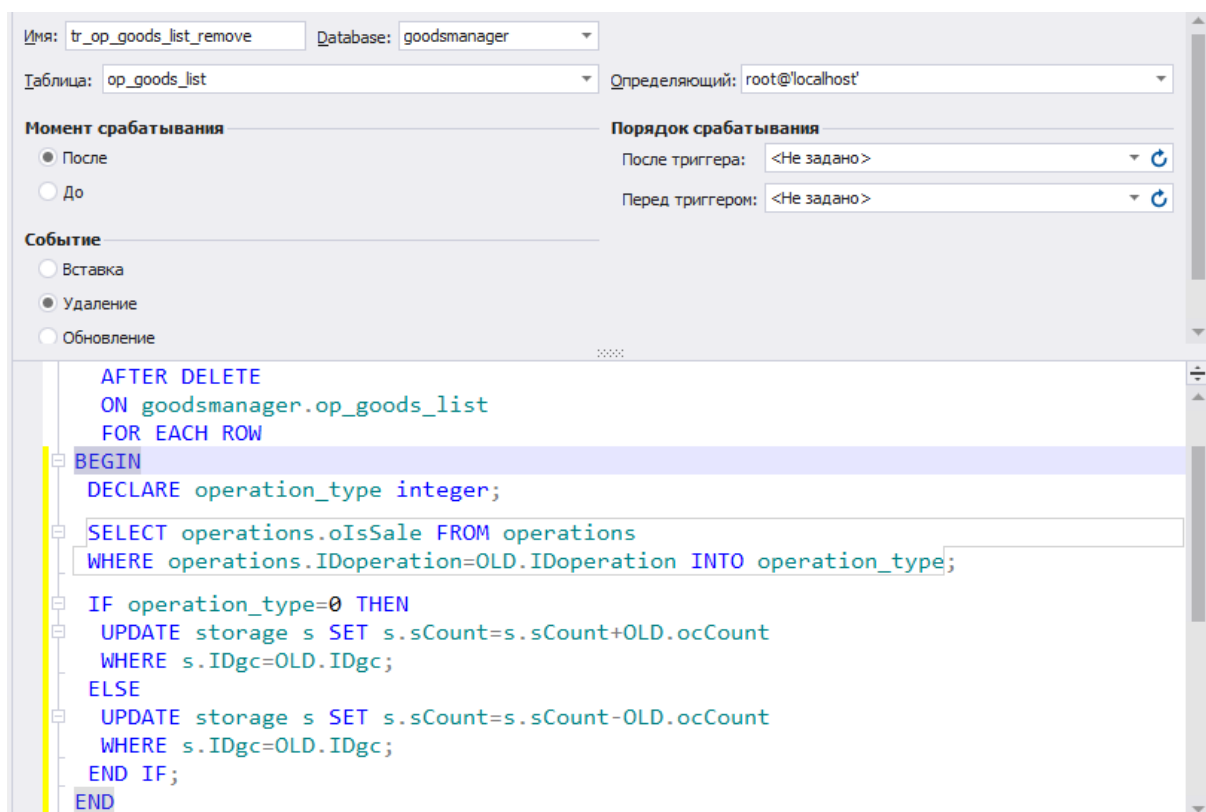


Рис. 4.2. Настройка триггера tr_op_goods_list_remove

Теперь остается реализовать триггер **tr_op_goods_list_edit**, который будет срабатывать после изменения какой-либо записи в таблице **op_goods_list** (рис. 4.3):

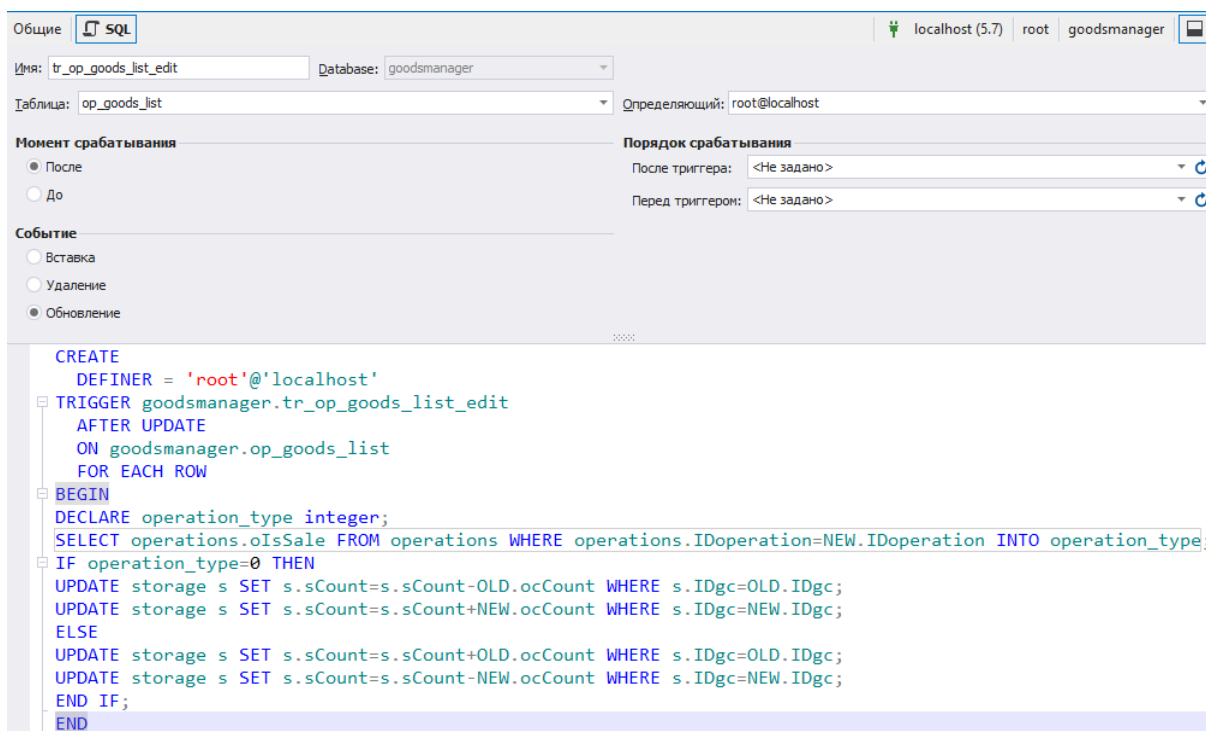


Рис. 4.3. Триггер на редактирование данных в op_goods_list

Для операции изменения нам доступны старые и новые значения поля **ocCount**. При модификации данных учитываются изменения нового значения относительно старого.

4.2 ПРОВЕРКА ВВЕДЕННОГО В ПОЛЕ ЗНАЧЕНИЯ

Рассмотрим средства проверки вводимых значений в разрабатываемом приложении. Для этого сделайте и откройте копию проекта **лабораторной работы №3**.

Для контроля вводимых данных будем использовать обработчики событий **OnSetText**, **OnValidate**, **OnChange** и **BeforePost**.

Проверить введенное в поле значение на его соответствие некоторым ограничениям или условиям можно в обработчике события **OnValidate**. Оно наступает при изменении значения поля вручную или программно до выполнения метода **Post**, сохраняющего изменения в БД.

Поэтому если полю присвоено неверное значение, то выполнение метода **Post** необходимо предотвратить с помощью метода **Abort** или вызовом исключительной ситуации (**raise Exception.Create**).

Выполним проверку поля **scEmail (T_suppliers_catalogscEmail)** таблицы **suppliers_catalog** на наличие символа **@**.

Настройка событий для полей доступна в инспекторе объектов после выбора поля в редакторе полей компонента (рис 4.4):

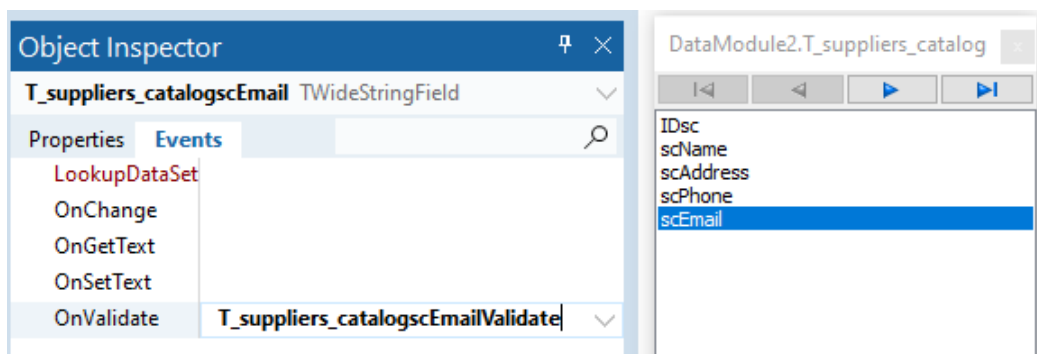


Рис. 4.4. Выбор события *OnValidate* для поля *scEmail*

Пропишем следующий код обработчика события *OnValidate*:

```
procedure TDataModule2.T_suppliers_catalogscEmailValidate(Sender:
  TField);
begin
  if not(pos('@', T_suppliers_catalogscEmail.AsString) > 0) then
  begin
    ShowMessage('В поле отсутствует символ @!');
    Abort;
  end;
end;
```

Чтобы работала процедура **ShowMessage** необходимо добавить в блок **uses** раздела **interface** исходного кода заголовочный файл **Dialogs** (в версиях начиная с ХЕ – **Vcl.Dialogs**).

Если символ **@** не содержится в значении, присвоенном полю, то метод **Abort** не позволит выполнить метод **Post** и запись с неверными данными не будет физически сохранена в БД. При этом НД останется в том состоянии, в котором он находился – в режиме редактирования **dsEdit** или добавления новой записи **dsInsert**.

Для проверки введенного значения может быть использовано и другое событие – **OnSetText**. Подобно событию **OnValidate**, оно возникает при изменении значения поля.

Особенностью события **OnSetText** является то, что в обработчик передается константа-параметр **Text**, содержащая в текстовом виде новое значение, назначенное полю, в то время как действительное значение поля остается без изменения.

Выполним проверку введенного значения в поле **gcCost** таблицы **goods_catalog**, которое должно быть выше некоторой минимальной цены, допустим 5 руб.:

```
procedure TDataModule2.T_goods_cataloggcCostSetText(Sender: TField;
  const Text: string);
var
  Tmp: double;
begin
  Tmp := StrToFloat(Text);
  if Tmp < 5 then
    ShowMessage('Стоимость товара должна быть больше 5 руб.')
  else
    T_goods_cataloggcCost.Value := Tmp;
end;
```

Функция **StrToFloat** конвертирует строку в значение с плавающей точкой.

Событие **OnChange** может быть использовано для тех же целей, что и событие **OnValidate**. Сделаем проверку вводимого количества товара в таблице список операций (**op_goods_list**), так значение поля **ocCount** должно быть не менее 1:

```
procedure TDataModule2.T_op_goods_listocCountChange(Sender: TField);
begin
  if T_op_goods_listocCount.Value < 1 then
    raise Exception.Create
      ('Необходимо указать количество не менее одной единицы товара');
end;
```

При изменении значения записи, события вызываются в следующем порядке:

– **OnSetText**;

- **OnValidate;**
- **OnChange.**

В качестве дополнительного средства проверки данных можно использовать обработчик события **BeforePost**. Это событие вызывается не для конкретного поля таблицы, а в целом для компонента **TADOTable** перед пересылкой любых изменений, выполненных в текущей записи.

Типичным применением процедуры обработчика событий **BeforePost** является реализация правил проверки для всей записи. Если запись не удовлетворяет проверке, то генерируется исключение, и некорректная запись не принимается.

Так в рамках рассматриваемой базы данных будем вести контроль товаров, участвующих в операции. В рамках контроля будем проверять выбран ли товар и в случае операции продажи корректно ли указано количество товара, которое не должно превышать запас на складе. Для оценки модификации будем использовать старые и новые значения полей **OldValue** и **NewValue** соответственно. Для проверки, содержит ли поле какие-либо данные, используется свойство **IsNull**, которое во время выполнения возвращает значение **True**, если поле содержит пустое значение.

```
procedure TDataModule2.T_op_goods_listBeforePost(DataSet: TDataSet);
var
  ErrMsgCnt: string; // для сообщения о недостаточном количестве товара
  mod_val: integer; // для расчета допустимого количества
begin
  ErrMsgCnt := 'Количество продаваемого товара должно быть не меньше
имеющегося на складе. Доступно для оформления продажи: ';
  if T_op_goods_listIDgc.IsNull or T_op_goods_listocCount.IsNull then
    raise Exception.Create('Необходимо выбрать наименования товара и ' +
' указать количество ');
  else if (T_operationsoIsSale.Value = 1) then
    begin
      if T_op_goods_list.State = dsInsert then
        if (T_op_goods_listocCount.Value >
          Q_v_selector_goodssCount.Value) then
          raise Exception.Create(ErrMsgCnt +
            Q_v_selector_goodssCount.AsString);
      if T_op_goods_list.State = dsEdit then
        begin
          mod_val := Q_v_selector_goodssCount.Value +
            T_op_goods_listocCount.OldValue;
          if (T_op_goods_listIDgc.OldValue <> T_op_goods_listIDgc.NewValue)
            and (T_op_goods_listocCount.Value >
              Q_v_selector_goodssCount.Value) then
            raise Exception.Create(ErrMsgCnt +
              Q_v_selector_goodssCount.AsString)
          else
            if (T_op_goods_listIDgc.OldValue = T_op_goods_listIDgc.NewValue)
```

```

        and (T_op_goods_listocCount.Value > mod_val) then
      raise Exception.Create(ErrMsgCnt + inttostr(mod_val));
    end;
  end;
end;

```

Помимо этого, в событии **BeforePost**, целесообразна проверка текстовых полей. Так, например, в нашем случае в каталоге товаров (**goods_catalog**) названия товаров должны быть не меньше **двух** символов, обязательно указан тип товара и установлена цена. В случае реализации этих проверок, в обработчик события **BeforePost**, набора данных **T_goods_catalog** запишите следующий код:

```

procedure TDataModule2.T_goods_catalogBeforePost(DataSet: TDataSet);
begin
  if T_goods_catalogIDtg.IsNull or T_goods_cataloggcCost.IsNull or
    (Length(T_goods_cataloggcName.AsString) < 2) then
    raise Exception.Create('Наименования товара, тип товара и ' +
      'цена не должны иметь пустые значения, а также Наименования ' +
      'товара иметь короткую запись (менее трех символов)');
  end;

```

Событие **BeforePost** также можно использовать для организации проверки и полей других типов. Например, у полей, указывающих диапазон, можно проверить, чтобы левая граница диапазона всегда была меньше правой, и т. п.

4.3 ФОРМАТИРОВАНИЕ ОТОБРАЖЕНИЯ ЗНАЧЕНИЙ ПОЛЕЙ

При отображении данных часто возникает задача модификации отображаемого значения для улучшения восприятия этих данных. Так, например, может потребоваться изменение структуры вывода значения (для даты – указать определенное расположение элементов, для числа – поставить разделители или ограничения на количество знаков после запятой) или определенное форматирование для значения (добавить дополнительную подпись к значениям, взять в кавычки, изменить регистр символов и т. п.). Для выполнения такого форматирования можно воспользоваться событием **OnGetText** со свойствами полей.

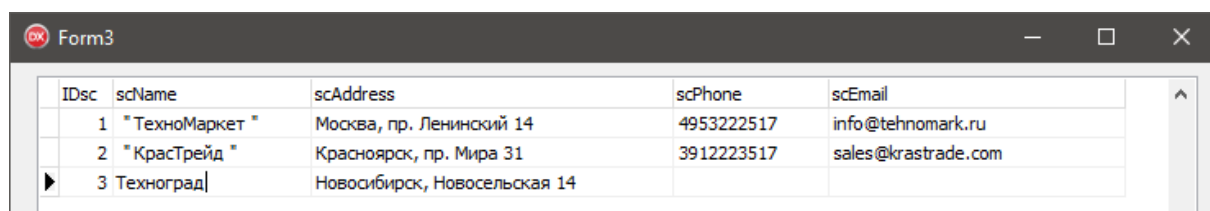
Рассмотрим работу с событием **OnGetText**. Пусть поле таблицы базы данных хранится не в том виде, в котором оно должно быть представлено. Тогда его можно отформатировать перед тем, как оно будет показано в визуальных компонентах, работающих с данными, определив алгоритм форматирования в обработчике события **OnGetText** компонента **TField**. В процедуре обработчика присутствуют следующие параметры:

- **Text** – выдает отформатированное значение, показываемое в визуальных компонентах, связанных с таблицами базы данных;

– **DisplayText** – определяет, когда произошло событие **OnGetText**, при показе поля (значение **True**) или при модификации поля (значение **False**).

Например, пусть поле **scName** из набора данных **T_SuppliersCatalog** отображается в компоненте **DBGrid1**. При показе содержимое данного поля необходимо заключить в кавычки (хотя хранится оно без кавычек). Для этого создадим обработчик (рис. 4.8):

```
procedure TDataModule2.T_suppliers_catalogscNameGetText(Sender:
  TField; var Text: string; DisplayText: Boolean);
begin
  if DisplayText then
    Text := ' ' + T_suppliers_catalogscName.AsString + ' '
end;
```



IDsc	scName	scAddress	scPhone	scEmail
1	"ТехноМаркет "	Москва, пр. Ленинский 14	4953222517	info@tehnomark.ru
2	"КрасТрейд "	Красноярск, пр. Мира 31	3912223517	sales@krastrade.com
3	Техноград	Новосибирск, Новосельская 14		

Рис. 4.8. Изменение отображения при помощи события *OnGetText*

Альтернативой для изменения формата отображения является свойство **DisplayFormat**. Однако если для поля был определен обработчик события **OnGetText**, то режимы форматирования, определенные в свойствах **DisplayFormat** и **EditMask** данного поля, игнорируются.

Свойству **DisplayFormat** присваивается текстовая строка, которая задает формат отображения (табл. 4.1).

Таблица 4.1

Спецификаторы форматов для числовых полей

Спецификатор	Описание
0	Число. Если незначащий разряд равен 0, то показывает его
#	Число. Если незначащий разряд равен 0, то не показывает его
.	Десятичная точка. Разделяет целую и дробную часть числа. Принимается во внимание только первая точка, остальные игнорируются
,	Разделитель тысяч. Каждая группа чисел из трех разрядов в целой части отделяется от иных разрядов запятой
;	Разделитель положительного, отрицательного и нулевого значения
E+	Научный формат действительных чисел
"xx" или 'xx'	Символы внутри двойных или одинарных парных кавычек не форматированы и выводятся как есть. Например, число 123.45 с форматом ## "рублей" выведется как '123.5 рублей'

К примеру, пусть значение поля равно **3456.777**, тогда если свойство **DisplayFormat** имеет значение **'#.##'**, то будет показано **3456.78**.

Теперь для **Q_v_goods_catalog**, отображающего информацию о товаре, добавим возможность изменения формата отображения поля **gcCost**. Для этого на соответствующей вкладке разместим компоненты **TEdit** с именем **E_dFormat** и **TButton** с именем **B_setFormat** (свойство **Caption** будет иметь значение: **Применить формат**) (рис 4.9):

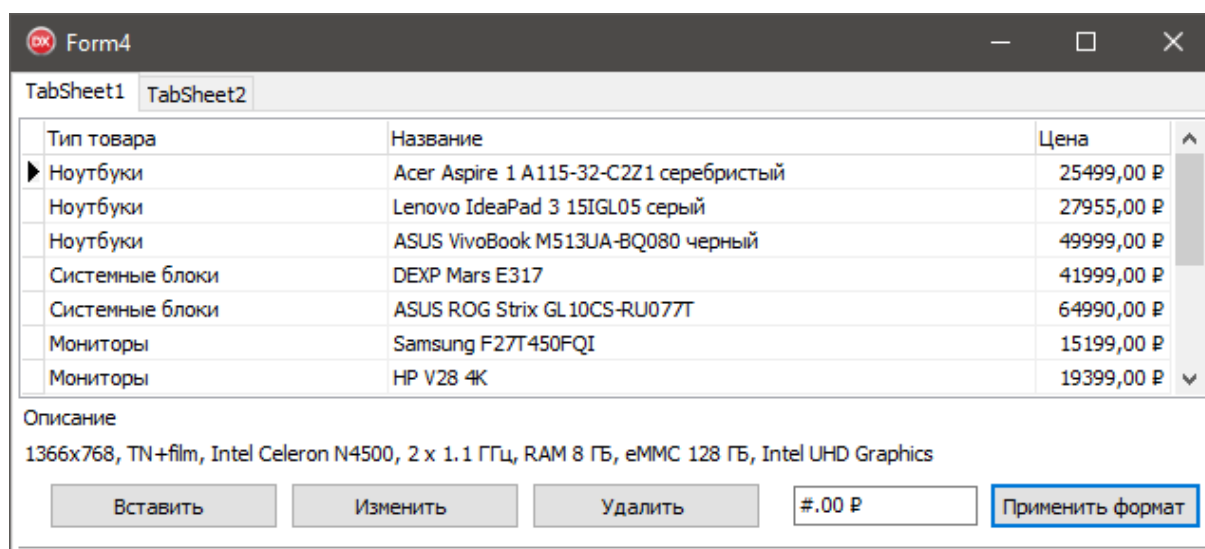


Рис. 4.9. Применение маски «#.00 P» к полю *gcCost*

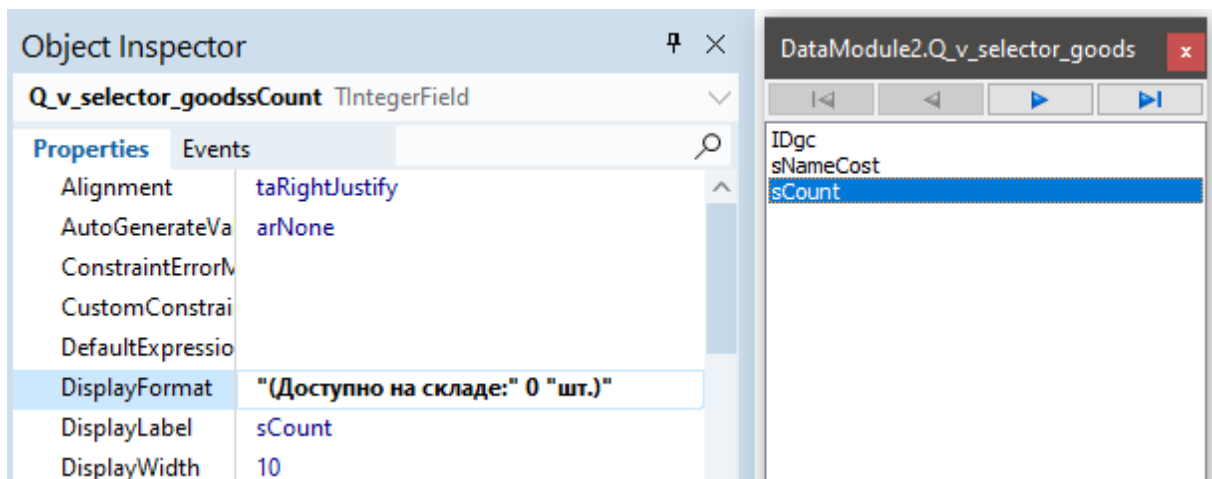
Для кнопки **B_setFormat** напишем обработчик события **OnClick**:

```
procedure TForm4.B_setFormatClick(Sender: TObject);
begin
DataModule2.Q_v_goods_cataloggcCost.DisplayFormat := E_dFormat.Text;
end;
```

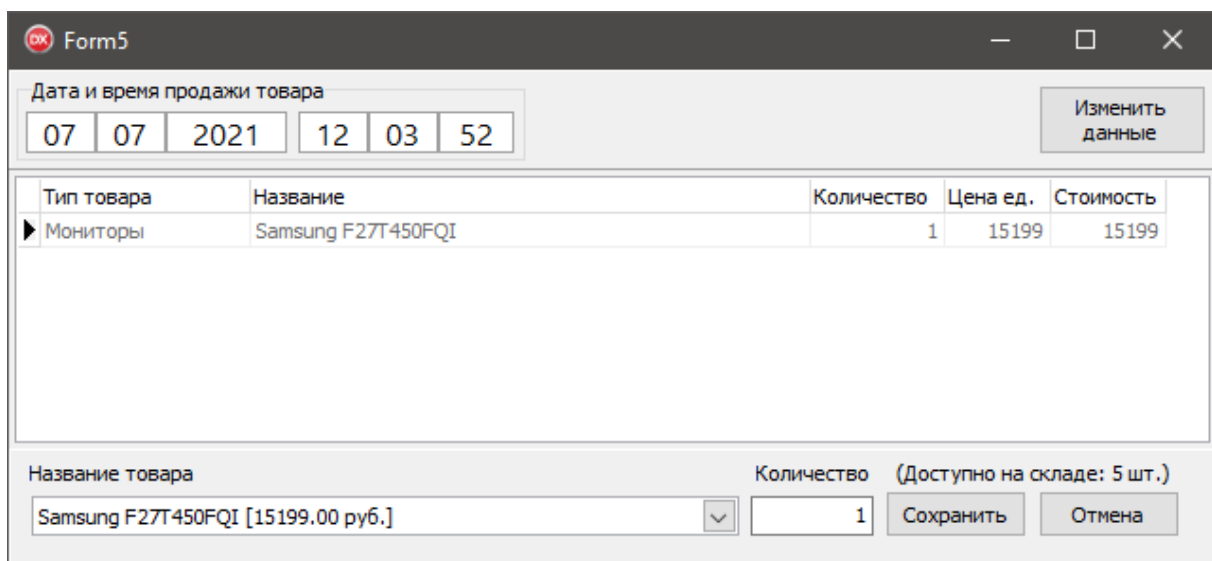
Свойству **Text** компонента **E_dFormat** зададим значение **#.00 P**.

При модификации состава операции целесообразно для выбранного товара отображать количество данного товара на складе. Изменим вид формы **Form5** для этого на панели **P_ModifyData** над кнопками **Сохранить** и **Отмена** разместим компонент **TDBText** (имя **DBText2**) и установим значение **DataModule2.DS_Q_v_selector_goods** свойству **DataSource**, а свойству **DataField** – значение **sCount**.

Теперь, используя свойство **DisplayFormat**, полю **sCount** зададим маску **"(Доступно на складе:" 0 "шт.)"** (Рис 4.10 а). После запуска программы на форме будут отображены сведения о доступном количестве товара на складе (рис 4.10 б).



а



б

Рис. 4.10. Пример использования свойства **DisplayFormat**: а – настройка свойства **DisplayFormat** поля **sCount**; б – результат работы.

Свойство **DisplayFormat** игнорируется, если определен обработчик для события **OnGetText**.

4.4 ФОРМАТИРОВАНИЕ ПОЛЕЙ ВО ВРЕМЯ ИХ РЕДАКТИРОВАНИЯ

Для контроля корректности вводимых значений применяется свойство **EditMask**.

Ограничения на вводимую информацию накладываются при помощи формата. При этом если введенный символ не удовлетворяет маске, то он не воспринимается. Маска представляет собой символьную строку, состоящую из трех частей:

- собственно маски;

– символа, определяющего, будут ли литералы (символ после указателя \) включаться в формируемое значение как его часть (значение 1) или не будут (значение 0);

– символа, используемого для представления пробела.

Маска представляет собой строку, состоящую из символов, которые приведены в табл. 4.2:

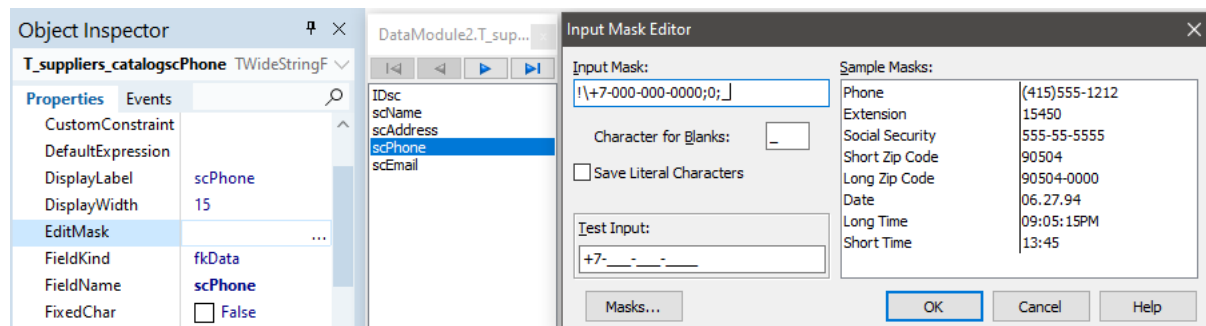
Таблица 4.2

Символы маски

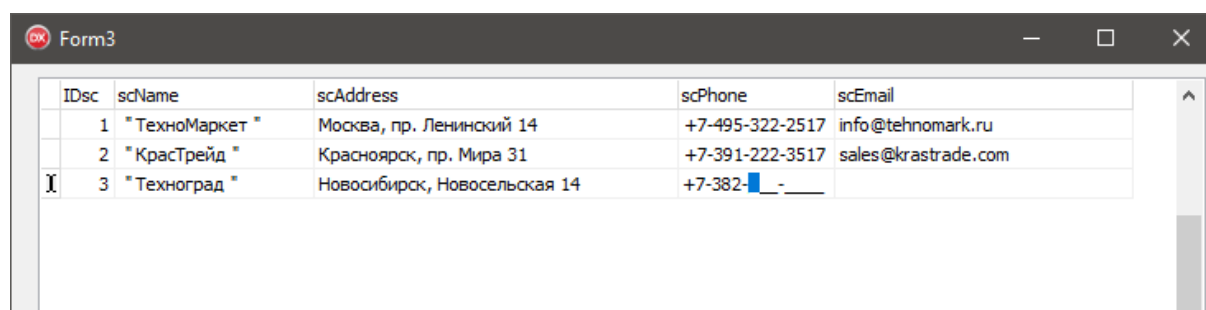
Символ	Описание
!	Подавляет ведущие пробелы. Если этот символ в данных отсутствует, то хвостовые пробелы подавляются
>	Все следующие символы будут на верхнем регистре, пока не встретится символ <
<	Все следующие символы будут на нижнем регистре, пока не встретится символ >
<>	Регистр не проверяется
\	Следующий за этим знаком символ является литералом, т. е. включается в формируемое значение
L	В этой позиции должен появиться только символ алфавита
l	Аналогично L, но символ в данной позиции может и отсутствовать
A	В этой позиции должен появиться только символ алфавита или цифра
a	Аналогично A, но символ в данной позиции может и отсутствовать
C	В позиции обязателен любой символ
c	Аналогично C, но символ в данной позиции может и отсутствовать
0	В данной позиции обязателен цифровой символ
9	В данной позиции может находиться цифра или она будет пустой;
#	В этой позиции должен появляться только цифровой символ, плюс или минус или не появляться никакой
:	Разделитель часов, минут и секунд для значения типа времени. Если в данной национальной кодировке для указанных целей используется иной символ, то он используется вместо символа :
/	Разделитель месяца, дня и года в датах. Если в данной национальной кодировке для указанных целей используется иной символ, то он используется вместо символа /
;	Разделитель частей маски
=	Заменитель пробела в маске

К примеру имеется маска '! (999\) 000\ -00\ -00;0;_ ' и введено значение '0952223344', то во время ввода это значение представлялось бы на экране как '(095) 222-33-44', а сохранилось бы в БД как '0952223344'.

Настроим маску для поля **scPhone** таблицы **suppliers_catalog**. Выберем нужное поле в редакторе полей, после чего нажмем на кнопку с многоточием у свойства **EditMask** и зададим маску для редактирования телефона поставщика в федеральном формате (рис. 4.11):



а



б

Рис. 4.11. Пример использования маски редактирования у поля **scPhone**: а – настройка; б – демонстрация работы

Для получения сведений о маске редактирования можно использовать свойство **EditMaskPrt**. его следует использовать вместо свойства **EditMask** в тех случаях, когда маска должна быть только прочитана. Также для форматирования значений полей перед их редактированием может быть использовано свойство **EditFormat**.

4.5 ФИЛЬТРАЦИЯ ДАННЫХ НА ОСНОВЕ SQL ЗАПРОСА

Важным механизмом для работы с данными является фильтрация/поиск информации.

Реализуем механизм фильтрации на вкладке **Состояние склада (Form1)**. Сначала разместим панель где будут расположены необходимые элементы (рис 4.12). У панели зададим свойство **Align** в значение **alTop** (чтобы панель была всегда сверху). На панели разместим один компонент **Label** для поясняющего текста и один компонент **Edit**.

Компоненту **Edit** дадим имя **E_Filter** и очистим его свойство **Text**, а свойству **TextHint** зададим значение: **Введите фрагмент названия товара**

или его типа. Свойство **TextHint** отвечает за вывод подсказки пользователю, если не введены какие-либо значения.

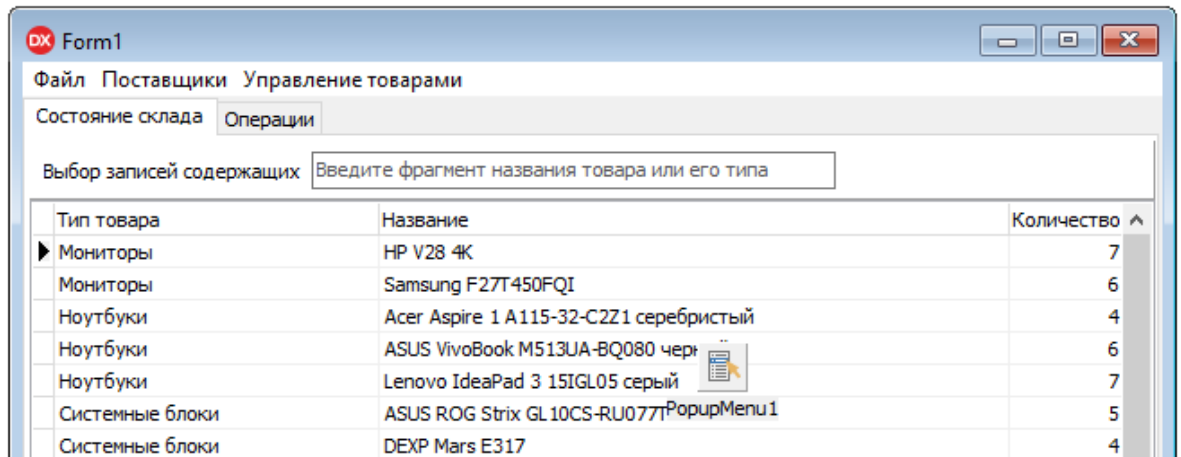


Рис. 4.12. Компоненты для фильтрации данных.

Для того чтобы фильтр применялся автоматически при вводе данных, напишем обработчик события **OnChange** для **E_Filter**:

```
procedure TForm1.E_FilterChange(Sender: TObject);
var
  baseSQL, filter_s: string;
begin
  DataModule2.Q_v_storage.DisableControls;
  DataModule2.Q_v_storage.Active := false;

  baseSQL := 'SELECT * FROM view_storage ';

  filter_s := '';
  if length(E_Filter.Text) > 0 then
    filter_s := 'WHERE tgName LIKE ' + QuotedStr('%' + E_Filter.Text +
      '%') + ' OR gcName LIKE ' + QuotedStr('%' + E_Filter.Text + '%');

  DataModule2.Q_v_storage.SQL.Text := baseSQL + filter_s;
  DataModule2.Q_v_storage.Active := true;
  DataModule2.Q_v_storage.EnableControls;
end;
```

Переменная **baseSQL** содержит основной запрос на выбор данных из представления **view_storage**, а переменная **filter_s** формирует строку с условием для отбора необходимых записей. Текстовые значения в **SQL**-выражениях заключены в одинарные кавычки, здесь для этих целей использована функция **QuotedStr**.

Запустив приложение и введя текстовый фрагмент для фильтрации, будут показаны только те товары, у которых в названии или в типе (названии типа товара) встречается введенный фрагмент (рис 4.13):

Form1

Файл Поставщики Управление товарами

Состояние склада Операции

Выбор записей содержащих As

Тип товара	Название	Количество
Ноутбуки	Acer Aspire 1 A115-32-C2Z1 серебристый	4
Ноутбуки	ASUS VivoBook M513UA-BQ080 черный	6
Системные блоки	ASUS ROG Strix GL10CS-RU077T	5

Рис. 4.13. Пример работы фильтра

Рассмотрим фильтрацию на основе SQL запроса применительно к отбору товаров по цене.

Для этого на форме **Form4** увеличим высоту панели **InsertEditDeletePanel** и разместим на неё **TGroupBox** (имя **GroupBox1**) для организации области фильтра (рис. 4.14 – 1). В области **GroupBox1** разместим компоненты **TComboBox** (зададим имя **ComboCostCondition**) (рис. 4.14 – 2), **TSpinEdit** (зададим имя **SpinCostValue**) (рис. 4.14 – 3), **TCheckBox** (зададим имя **ChkCostFilter**) (рис. 4.14 – 4) и два компонента **TLabel** для вывода поясняющего текста.

Form4

TabSheet1 TabSheet2

Тип товара	Название	Цена
Ноутбуки	Acer Aspire 1 A115-32-C2Z1 серебристый	25499
Ноутбуки	Lenovo IdeaPad 3 15IGL05 серый	27955
Ноутбуки	ASUS VivoBook M513UA-BQ080 черный	49999
Системные блоки	DEXP Mars E317	41999
Системные блоки	ASUS ROG Strix GL10CS-RU077T	64990
Мониторы	Samsung F27T450FQI	15199
Мониторы	HP V28 4K	19399

Описание
1366x768, TN+film, Intel Celeron N4500, 2 x 1.1 ГГц, RAM 8 ГБ, eMMC 128 ГБ, Intel UHD Graphics

Вставить Изменить Удалить #.00 Р Применить формат

Фильтр по цене

Условие > Значение 0 Включить

Тип товара Ноутбуки Название Acer Aspire 1 A115-32-C2Z1 серебристый Цена 25499

Описание 1366x768, TN+film, Intel Celeron N4500, 2 x 1.1 ГГц, RAM 8 ГБ, eMMC 128 ГБ, Запомнить Отменить

Рис. 4.14. Компоненты для фильтрации данных по цене

Для начала настроим компонент **ComboCostCondition**, в свойстве **Style** установим значение **csDropDownList**. Через редактор (рис 4.15) в свойстве **Items** зададим операторы сравнения: **>**, **>=**, **=**, **<>**, **<**, **<=** (каждый на новой строке). После определения списка в свойстве **ItemIndex** установим значение **0**.

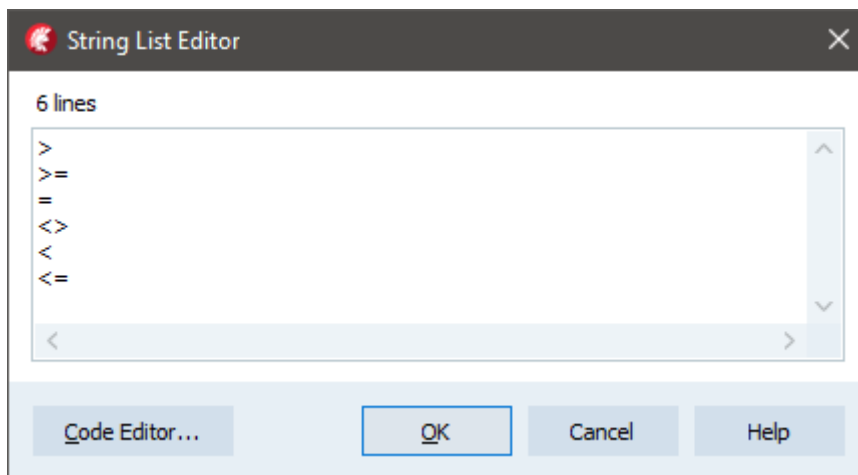


Рис. 4.15. Настройка списка условий фильтрации

Теперь напишем код фильтрации. Фильтр будет формироваться и применяться для компонента **ChkCostFilter** в обработчике события **OnClick**:

```
procedure TForm4.ChkCostFilterClick(Sender: TObject);
var
  baseSQL, filter_s: string;
begin
  DataModule2.Q_v_goods_catalog.Active := False;
  baseSQL := 'SELECT * FROM view_goods_catalog';
  DataModule2.Q_v_goods_catalog.SQL.Text := baseSQL;
  filter_s := '';
  if ChkCostFilter.Checked then
    begin
      // Формирования условия выборки
      filter_s := ' WHERE gcCost ' + ComboCostCondition.Text + ' ' +
        IntToStr(SpinCostValue.Value);
      DataModule2.Q_v_goods_catalog.SQL.Text := baseSQL + filter_s;
    end;
  DataModule2.Q_v_goods_catalog.Active := True;
end;
```

Для того чтобы фильтр автоматически обновлялся при изменении значения цены в компоненте **SpinCostValue**, а также выборе условия в **ComboCostCondition** необходимо имитировать наступление события **OnClick** компонента **ChkCostFilter**. Для этого пропишем код обработчи-

ков события **OnChange** у компонентов **ComboCostCondition** и **SpinCostValue**:

```
procedure TForm4.ComboCostConditionChange(Sender: TObject);
begin
    ChkCostFilterClick(nil);
end;

procedure TForm4.SpinCostValueChange(Sender: TObject);
begin
    ChkCostFilterClick(nil);
end;
```

Помимо использования модификации **SQL** запроса возможно осуществление фильтрации с использованием свойства **Filter** у **ADO** компонентов.

4.6 СВОЙСТВО FILTER ADO-КОМПОНЕНТОВ

Свойство **Filter** позволяет указать условия фильтрации. В этом случае НД будет отфильтрован, как только его свойство **Filtered** станет равным значению **True**. Синтаксис фильтра похож на синтаксис предложения **WHERE** **SQL**-оператора **SELECT** с тем исключением, что в фильтре указываются не имена переменных программы, а имена полей и литералы, т. е. явно заданные значения.

Строку фильтрации можно ввести во время выполнения приложения. В качестве примера рассмотрим фильтрацию поля содержащего дату по диапазону. Реализуем механизм фильтрации операций по диапазону дат.

Для этого на панель вкладки **Операции** главной формы поместим **TGroupBox** на который разместим один компонент **TCheckBox**, два компонента **TDateTimePicker** у которых будут имена **DateTimePicker1** и **DateTimePicker2** и компоненты **TLabel** (необходимы для поясняющего текста). Компонент **DateTimePicker1** будет отвечать за начальную дату интервала, а **DateTimePicker2** – за конечное (рис 4.16):

Дата операции	Тип операции	Стоимость	Название поставщика
06.05.2021 11:23:00	Продажа	549674	
05.05.2021 2:22:00	Продажа	129287	
07.05.2021 11:15:00	Поставка	397287	ТехноМаркет

Рис. 4.16. Фрагмент формы с компонентами для организации фильтра по дате

Для компонентов **DateTimePicker1** и **DateTimePicker2** в свойстве **Time** можно выставить значения **0:00:00** и **23:59:59** соответственно. Это необходимо, когда фильтрация будет осуществляться по полю **DateTime** (дата и время) с задействованием времени. В нашем примере будет происходить преобразование формата, которое отсекает время, поэтому данное действие не обязательно.

Активация фильтра будет осуществляться с применением события **OnClick** компонента **CheckBox1**. Обработчик события будет иметь следующий код:

```
procedure TForm1.CheckBox1Click(Sender: TObject);
begin
    // Отключение фильтра
    DataModule2.Q_v_goods_operations.Filtered := false;
    // Формирование строки фильтрации
    DataModule2.Q_v_goods_operations.Filter := 'oDateTime >= ' +
        QuotedStr(FormatDateTime('YYYY-MM-DD', DateTimePicker1.Date)) +
        ' and oDateTime <= ' + QuotedStr(FormatDateTime('YYYY-MM-DD',
        DateTimePicker2.Date));
    // задание параметра активности фильтра как у состояния CheckBox1
    DataModule2.Q_v_goods_operations.Filtered := CheckBox1.Checked;
end;
```

В коде применяется приведение формата даты при помощи функции **FormatDateTime** к конструкции **'YYYY-MM-DD'**, которая является стандартной для представления даты во внутреннем формате *MySQL*.

Поскольку поле **oDateTime** содержит дату и время, а в представленном коде события **OnClick** компонента **CheckBox1** идет использование информации только о дате, то значения будут выбираться по выбранную дату без её включения. Если необходимо включать конечную дату, то необходимо вести учет времени. Строку назначения фильтра в таком случае адаптировать для учета времени. Для этого время, в функции **FormatDateTime**, можно непосредственно прописать вручную:

```
DataModule2.Q_v_goods_operations.Filter := 'oDateTime >= ' +
QuotedStr(FormatDateTime('YYYY-MM-DD', DateTimePicker1.Date)) +
' and oDateTime <= ' + QuotedStr(FormatDateTime('YYYY-MM-DD 23:59:59',
DateTimePicker2.Date));
```

или использовать полное форматирование даты и времени:

```
DataModule2.Q_v_goods_operations.Filter := 'oDateTime >= ' +
QuotedStr(FormatDateTime('YYYY-MM-DD', DateTimePicker1.Date)) +
' and oDateTime <= ' + QuotedStr(FormatDateTime('YYYY-MM-DD HH:MM:SS',
DateTimePicker2.DateTime));
```

Поскольку в режиме активной фильтрации пользователь может изменить дату, то необходимо прописать код обрабатывающий данную ситуацию в компонентах **TDateTimePicker** используя событие **OnChange**:

```

procedure TForm1.DateTimePicker1Change(Sender: TObject);
begin
    CheckBox1Click(nil);
end;

procedure TForm1.DateTimePicker2Change(Sender: TObject);
begin
    CheckBox1Click(nil);
end;

```

В данных обработчиках происходит имитация активации события **OnClick** для компонента **CheckBox1** и тем самым фильтр перестраивается с учетом измененных данных в компонентах **TDateTimePicker**.

КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Дайте характеристику триггерам, каково их назначение?
2. Что из себя представляют псевдонимы **OLD** и **NEW**?
3. В какой момент срабатывают триггеры?
4. Опишите назначение конструкции **DECLARE**?
5. Для чего применяется функция **StrToFloat**, какие подобные функции существуют в **Delphi**?
6. Чем событие **BeforePost** отличается от **OnSetText**, **OnValidate**, **OnChange**?
7. Охарактеризуйте свойство **DisplayFormat**, для чего оно применяется?
8. Влияет ли формат вводимых данных в **EditMask** на сохраняемые значения в базе данных?
9. За что отвечает свойство **TextHint**?
10. Каким образом осуществляется фильтрация набора данных с использованием **SQL**-запроса?
11. Какие операторы отношения и логические операторы можно использовать при организации фильтрации?
12. Опишите механизм работы **SQL**-конструкции **LIKE**, каков формат применяемых масок?
13. За что отвечает **SQL**-конструкция **WHERE**?
14. Какие изменения будут происходить с НД при установке свойства **Filtered** в значение **True**?
15. Чем компонент **SpinEdit** отличается от **Edit**?
16. Для чего применяются функции **FormatDateTime** и **QuotedStr**?
17. Какой формат хранения даты в **MySQL**?